Bill Bloom

(React Fundamentals & Component Architecture)

Hello learner!

In the previous phase, you focused on building the backend foundation of *Bill Bloom*. You designed models, implemented APIs, and structured your application logic.

Now, it's time to bring your application to life.

Background

So far, your application has no user interface. All interactions happen through APIs, making it difficult for users to visualize and interact with their financial data.

This week, you will build the **frontend layer** of the application using React. Instead of building full pages with routing, you will focus on **creating reusable, independent UI components** that represent the core building blocks of the application.

These components will simulate real-world behavior using **dummy data**, preparing you for full-stack integration in the upcoming weeks.

Problem Statement

Your task this week is to build a **component-driven React frontend** for the Bill Bloom application.

You must:

- Set up a React project using Vite
- Build reusable, self-contained UI components
- Implement forms using controlled components

- Use props and state for data flow and interactivity
- Implement conditional rendering for dynamic UI behavior
- Simulate application data using a centralized dummy data file

By the end of this week, your frontend will have a template UI that can show various pages.

---

1. Project Setup

Create a React project(bill-bloom-frontend) using Vite and organize your project with a clean and scalable folder structure.

Note: *Include CSS files to style the application as and where needed. Keep the styles minimal, as we will use bootstrap styles next week.*

```
src
  data
    dummyData.js
  components
    layout
      Header.jsx
      Footer.jsx
    auth
      LoginForm.jsx
      RegisterForm.jsx
   groups
      GroupCard.jsx
      GroupList.jsx
      CreateGroupForm.jsx
      GroupDetail.jsx
    expenses
      PersonalExpenseForm.jsx
      GroupExpenseForm.jsx
      PersonalExpensePage.jsx
  pages
      AuthPage.jsx
      GroupDetailPage.jsx
      GroupsPage.jsx
      HomePage.jsx
      PersonalExpensesPage.jsx
```

---

2. Dummy Data (Single Source of Truth)

Create a **dummyData.js** file that mirrors your backend models. This file will act as your **temporary database**. Include Users, Groups, Personal Expenses, Group Expenses, and Settlements. Ensure the structure closely matches backend schemas so future integration becomes seamless.

---

3. Layout Components

A. Header Component

- Displays app name: *Bill Bloom*
- Dummy logged-in user section

B. Footer Component

- Displays copyright
- App tagline

---

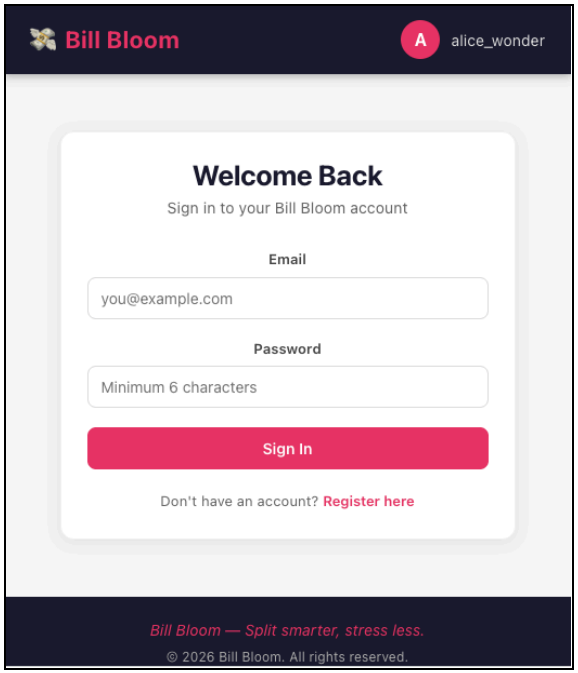
4. Authentication Components

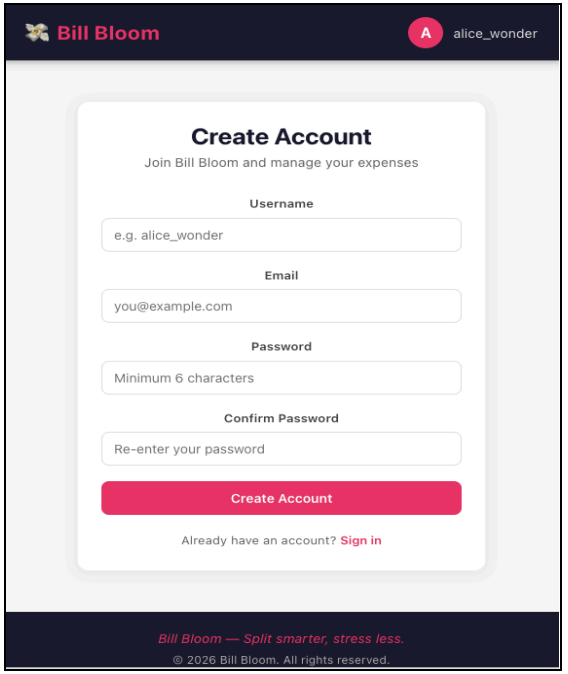
A. Login Form

- Create a form with email and password inputs.
- Validate the email format and ensure the password has at least 6 characters.
- On submit, prevent the default behavior, log the form data to the console, and show a success message.
- Use conditional rendering to display validation errors.

B. Register Form

- Create a form with username, email, password, and confirm password inputs.
- Validate that all fields are filled, the email format is correct, the password meets requirements, and both passwords match.
- On submit, prevent default behavior, log the data, show a success message, and simulate a redirect to the login page.
- Use conditional rendering to display validation errors.

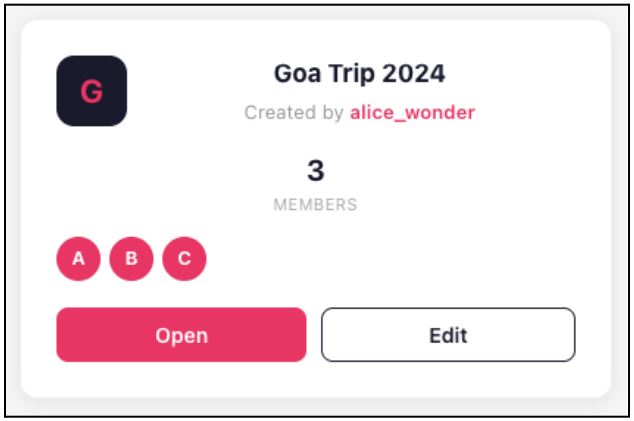




5. Group Components

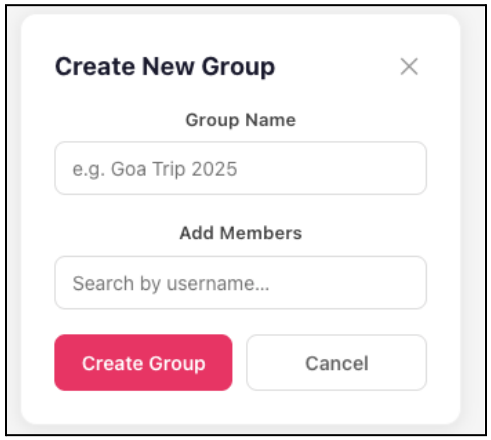
A. Group Card

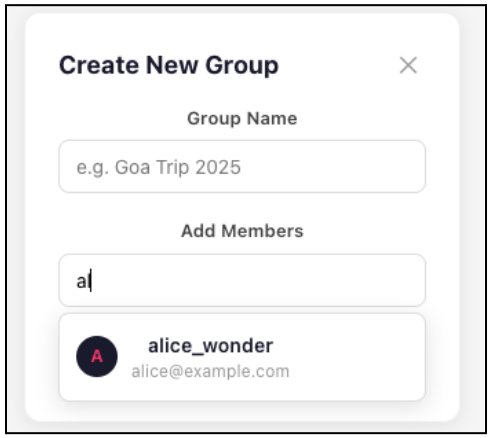
- Create a GroupCard component that displays the **group name**, **member count**, and **creator name**.
- Include "Open" and "Edit" buttons.

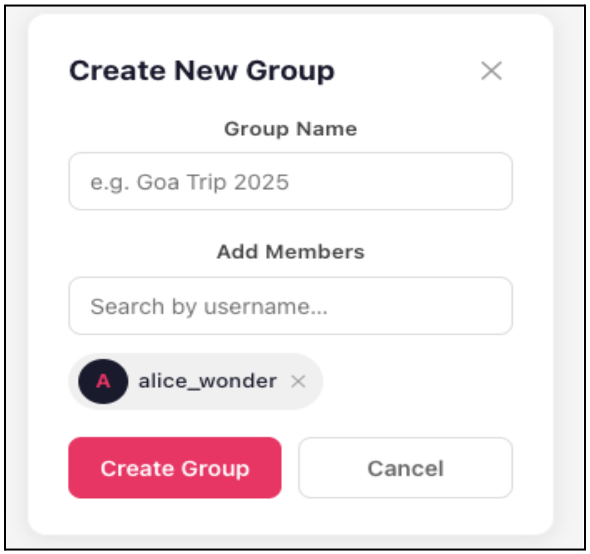


B. Create Group Form

- Create a group form that takes a **name** and **group members** as input.
- Use a search input field to search for users and select from the dropdown result.
- Ensure the frontend validations are performed.
- On submit, log the name and member IDs as { name, memberIds }.







---

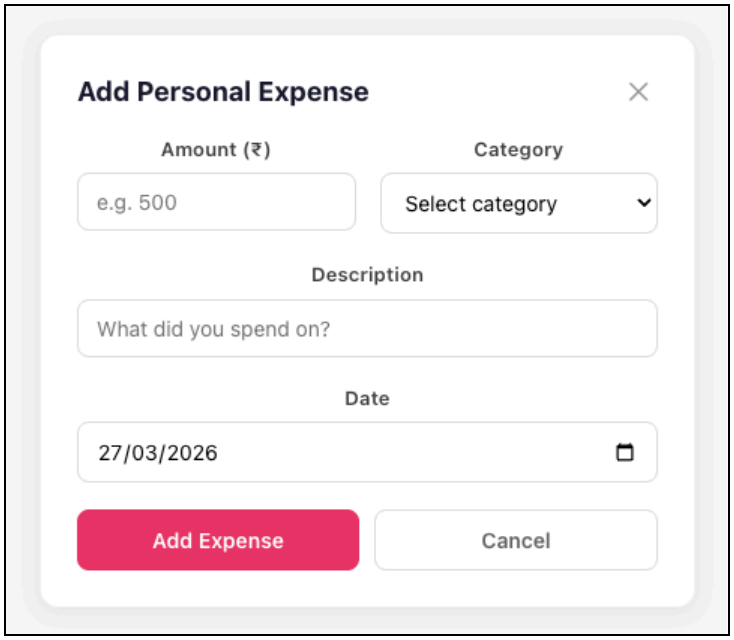
6. Expense Components

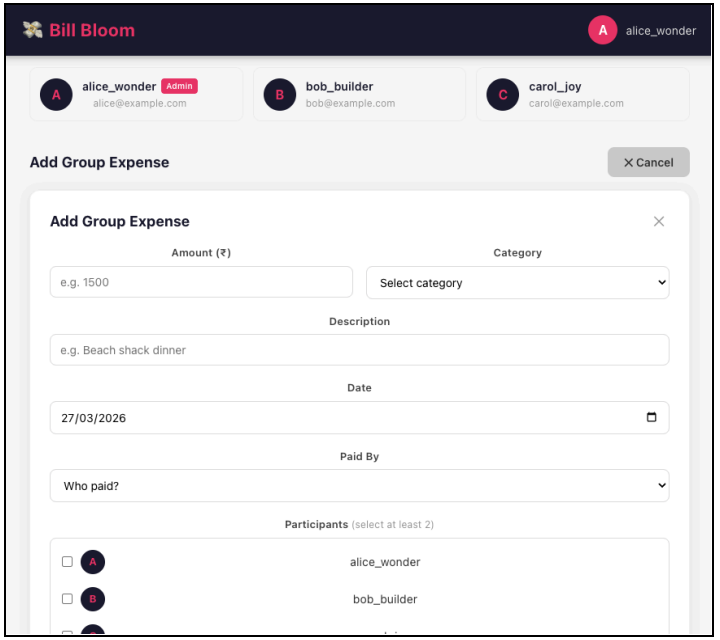
A. Personal Expense Form

- Create a form with inputs for amount, category, description, and date.
- Validate that the amount is greater than 0.
- On submit, log the data and trigger a callback function.

B. Group Expense Form

- Create a form with inputs for amount, category, description, and date, along with a paidBy dropdown and a participants multi-select with checkboxes.
- Validate that a payer is selected and at least two participants are chosen.
- Use props to populate members





7. Pages

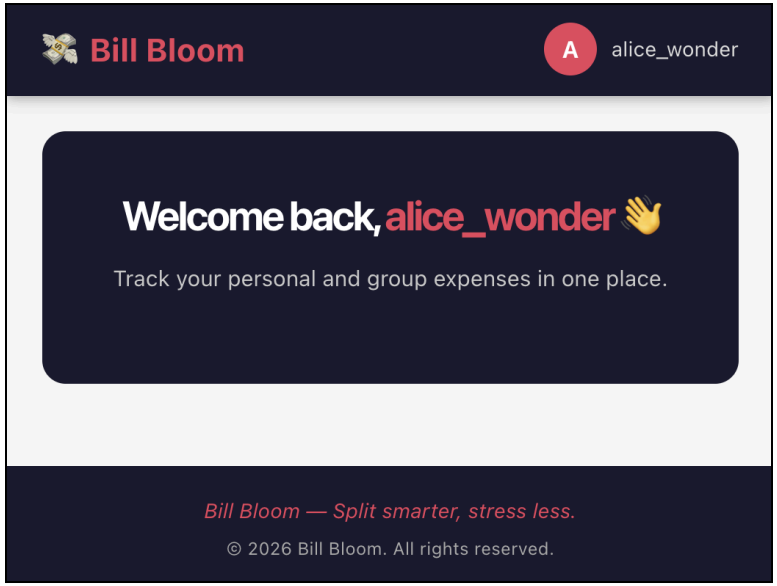
A. Auth Page

Create an **Auth Page** component that manages a state to toggle between authentication forms.

- Initialize the view state using the **page** prop.
- Render the **LoginForm** and **RegisterForm** components as per the prop value.
- Display a fallback message for any invalid view value.

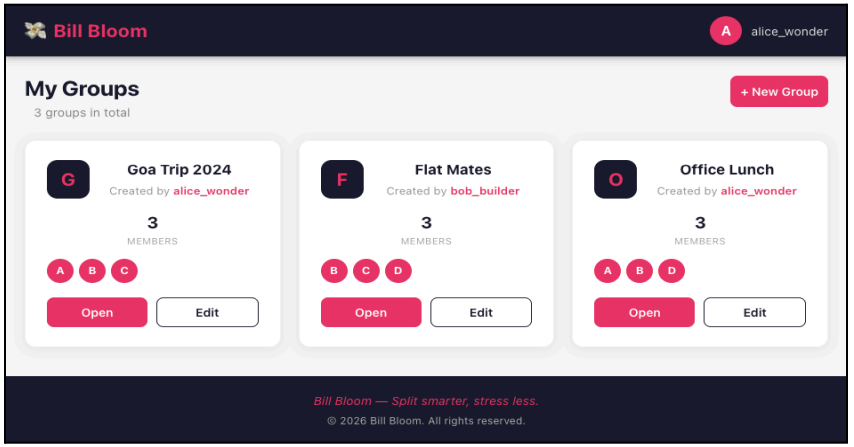
B. Home Page

- Create a simple **Home Page** component that displays a message as shown below.
- We will work on this in upcoming weeks.



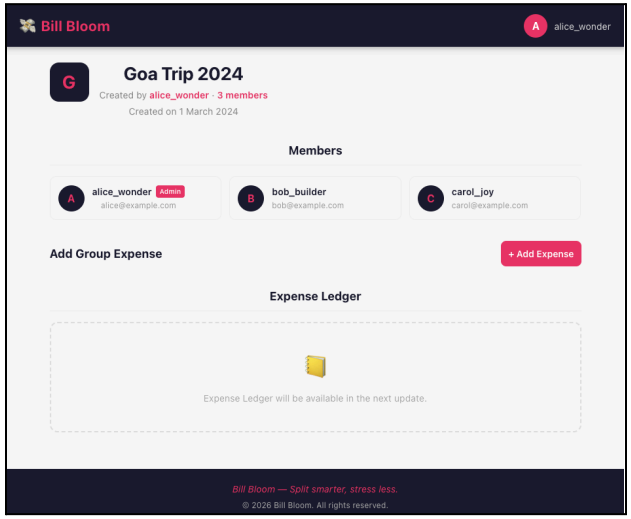
C. Groups Page

- Create a Groups Page component that displays all groups using the GroupCard component.
- Include functionality to toggle the Create Group Form.



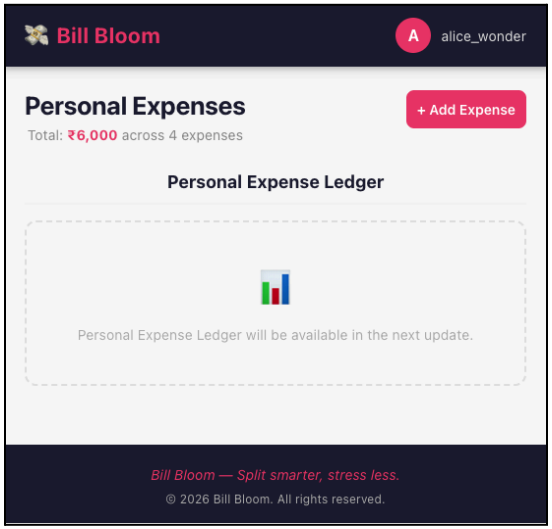
D. Group Detail Page

- Create a Group Detail component that displays the group name, list of members, and creator information.
- Include a toggle to show the GroupExpenseForm.
- Display a placeholder message: *"Expense Ledger will be available in the next update."*



E. Personal Expense Page

- Create a component that maintains expense data and form visibility in the state.
- Provide functionality to toggle the form and add a new expense to the state.
- Display a placeholder message: *"Personal Expense Ledger will be available in the next update."*



---

8. Development Approach (App.jsx)

- We have built components(cater to various pages) and pages(cater to features).
- Import these pages into App.jsx one at a time and test them.

Happy Coding